DTPN: A Diffusion-based Traffic Purification Network for Tor Website Fingerprinting

Chenchen Yang
Shenzhen International Graduate School,Tsinghua University
Shenzhen, China
Key Laboratory of Cyberspace Security,Ministry of Education of China
Zhengzhou, China
Peng Cheng Laboratory
Shenzhen, China
orcayang@foxmail.com

Xi Xiao*
Shenzhen International Graduate School,Tsinghua University
Shenzhen, China
Key Laboratory of Cyberspace Security,Ministry of Education of China
Zhengzhou, China
Peng Cheng Laboratory
Shenzhen, China
xiaox@sz.tsinghua.edu.cn

Guangwu Hu
Shenzhen Institute of Information Technology
Shenzhen, China
hugw@sziit.edu.cn

Zhen Ling
Southeast University
Nanjing, China
zhenling@seu.edu.cn

Hao Li
National Key Laboratory of Advanced Communication Network
Shijiazhuang, China
cuclihao@163.com

Bin Zhang
Peng Cheng Laboratory
Shenzhen, China
bin.zhang@pcl.ac.cn

Abstract

Website Fingerprinting attack is a type of method used to classify network traffic generated by users on the Tor (The Onion Router) based on the websites they visit, leading to the leakage of individuals' privacy . For Website Fingerprinting attack, network traffic defense methods involve adding noise to the original network traffic to render the attacker's methods ineffective. Previous attack methods primarily focused on improving classification accuracy by enhancing the attack model, with adversarial training being the most common approach. However, adversarial training requires frequent updates and exhibits poor generalization when dealing with previously unseen network traffic protection methods. In order to address the limitations of adversarial training, a novel method is proposed leveraging a diffusion model for network traffic purification. This paper is the first to use a diffusion model to resist network traffic defense based on adversarial perturbations. The diffusion models are theoretically suited for data purification in the training mode, i.e., removing noises generated by adversarial perturbations from the data. Our method enables existing network traffic classification methods to maintain effective classification of network traffic after protection without requiring retraining, while also achieving good generalization performance with previously unseen network traffic defense methods. The purified network traffic data can effectively improve the robustness of existing website fingerprinting methods. Experiments conducted under various network traffic defense strategies demonstrate that the proposed method increases accuracy by up to 60.8% on DF dataset and 50.3% on CW100 dataset, respectively, compared to adversarial training.

CCS Concepts

• **Security and privacy** → *Software and application security*; Web application security;

Keywords

Website Fingerprinting; adversarial training; diffusion model; traffic purification

ACM Reference Format:
Chenchen Yang, Xi Xiao, Guangwu Hu, Zhen Ling, Hao Li, and Bin Zhang. 2025. DTPN: A Diffusion-based Traffic Purification Network for Tor Website Fingerprinting. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (WSDM '25), March 10–14, 2025, Hannover, Germany*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3701551.3703547

*Corresponding author

This work is licensed under a Creative Commons Attribution International 4.0 License.

WSDM '25, March 10–14, 2025, Hannover, Germany
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1329-3/25/03
https://doi.org/10.1145/3701551.3703547

1 INTRODUCTION

Research on network traffic includes techniques such as traffic prediction[1]and traffic classification, aimed at enhancing data management and privacy protection. Traffic classification, in particular, is a form of privacy mining, as it involves extracting insights from network data while potentially exposing sensitive information. Additionally, web traffic and log analysis play a crucial role in understanding user behaviors and improving website performance, while ensuring that user privacy is maintained.

Tor (The Onion Router) [2] is an anonymous communication network designed to enable users to browse and transmit data on the Internet while preserving their anonymity, including that of their web traffic. However, despite its ability to offer substantial

privacy, the traffic generated by Tor users remains vulnerable to a type of attack called Website Fingerprinting (WF).

Prior WF methods [3–7] often rely on hand-crafted features for performing WF attacks. However, current methods [8–10] predominantly employ deep learning-based methods. These methods eliminate the cumbersome process of feature engineering, accelerate the deployment of WF attacks.

Network Traffic Protection (NTP) methods combat WF attacks by introducing noise into the original data flow.This strategy includes adding spurious packets, altering packet content, or introducing delays to undermine the attacker's methods. Existing measures for NTP fall into two principal categories: traffic obfuscation [11–15] and adversarial samples [16–20]. Traffic obfuscation adds noise at the application or network layer but often results in high computational overhead or inadequate protection against specific traffic classifications. The most effective current defense involves adversarial perturbations, where minimal noise is introduced to Deep Neural Networks (DNNs) during training, leading WF classifiers to misclassify traffic.

Current methods for network traffic classification use neural networks, which are vulnerable to adversarial attacks. These attacks exploit minor input changes to deceive the network into making incorrect predictions, reducing the effectiveness of neural networks in WF attacks. To bypass NTP defenses, WF attackers use Adversarial Training, which targets specific inputs and is impractical for full network flows. This strategy struggles with frequent updates and new protection methods, and its limitations against protections like AdvTraffic [19] have been demonstrated.

To address the challenge, we introduce a novel network traffic purification method centered on generative models in this study. Our proposed method empowers established network traffic classification methodologies to proficiently categorize safeguarded network traffic without necessitating retraining efforts. Furthermore, it demonstrates robust generalization capabilities for novel NTP methodologies that have not been previously encountered. The key contributions of our work are as follows:

- We pioneer the application of diffusion models in WF and introduce the concept of traffic purification to boost WF attack capabilities. Our Diffusion-based Traffic Purification Network (DTPN) [1] model is the first one proposed and applied in the field of traffic purification.
- Drawing inspiration from the UNet architecture, we combine TransformerBlock and one-dimensional convolution to design a sophisticated UNetFormer, which is utilized as a noise estimator in our introduced diffusion model. Furthermore, in consideration of the unique requirements of our task, we introduce a novel loss function to enhance the generalization of model.
- We use DTPN to obtain purified traffic and apply it to test WF attack methods, achieving significantly better results. In experiments conducted on two datasets, we achieve a maximum improvement of 60.8% and 50.3% compared to attacks without traffic purification.

[1]Available at https://github.com/CodeWGame/DTPN

2 RELATED WORK

2.1 WF Attacks

Starting from the initial proposal by Herrmann et al. to use the Naive Bayes method for WF attacks on Tor [3], the use of machine learning-based approaches becomes more prevalent. For instance, in [4], [21], and [5], SVM and KNN are respectively employed for classification. With the advancement of deep learning techniques, Sirinam et al. [8] developed Deep Fingerprints (DF) for Tor browsers based on complex Convolutional Neural Networks (CNNs). Subsequently, in [22] and [10], the classifiers are also enhanced based on DF. Additionally, Bhat et al. [9] introduced Var-CNN for WF using the ResNet-18 architecture.

2.2 WF Defenses

Traffic Obfuscation. These defense mechanisms are designed to make network traffic more difficult to classify. Early defense measures [11, 12] used constant-rate padding to reduce information leakage caused by timing intervals and traffic patterns. However, these methods resulted in significant bandwidth overhead, exceeding 150%. Recently, research efforts have aimed to reduce this overhead by inserting packets at positions with large time intervals between packets (WTF-PAD [13]). However, WTF-PAD achieved protection success rates of only 9%, when facing strong DNN-based network traffic classification attacks. Supersequence Defense [14, 15] attempted to find a supersequence, which is a long sequence of network flows containing subsequences from different website network flow sequences. The most effective supersequence defense, Walkie-Talkie, achieved a 50% success rate in protecting against DNN attacks.

Adversarial Perturbations. Goodfellow et al. first introduced evasion attacks against DNNs, where adversaries introduce small adversarial perturbations [23] to the input of a DNN, causing it to misclassify the input. Recent research on NTP [16, 17] has used adversarial perturbations to counter DNN-based attacks. However, adversarial perturbation-based NTP faces the challenge of requiring prior knowledge of the entire future network flow to compute the necessary adversarial perturbation before establishing a connection. Mockingbird [17] addressed this limitation by calculating adversarial perturbations using a collected network traffic database and applying them to new network traffic. Meanwhile, Nasr et al. [18] proposed precomputing universal adversarial perturbations for unknown network traffic to achieve real-time protection. However, attackers can enhance the robustness of their models through adversarial training [24, 25], leading to a significant drop in the protection rate to 66.7% [19]. Furthermore, Minipatch [20] reduced bandwidth overhead by employing adversarial patches. AdvTraffic [19] utilized AdvGAN [26] for adversarial training against WF attackers, and experimental results indicated that adversarial training is no longer effective for AdvTraffic.

2.3 Diffusion Models

The generative model is a type of model that can stochastically generate observed values based on latent parameters. In recent years, notable diffusion-based generative models have been proposed, including diffusion probabilistic models [27], Noise Conditional

Score Networks (NCSN) [28], and Denoising Diffusion Probabilistic Models (DDPM) [29]. These models have been applied across various fields, including recommendation[30], knowledge graphs[31], and so on. DiffPure [32] proposes a diffusion model to undermine adversarial samples in computer vision. With the diffusion forward and reverse processes, such small noises can be removed. The state-of-the-arts of WF defenses are based on adversarial samples (perturbations). Inspired by DiffPure, we leverage the diffusion model to remove the perturbations in Tor burst sequence.

3 METHODOLOGY

The objective of this study is to perform a purification operation on protected traffic that utilizes the WF defense method by removing the noise introduced by this defense method. The overall architecture of our method, the Diffusion-based Traffic Purification Network (DTPN), is shown in Figure1. Table 1 provides a comprehensive list of notations used in this paper.

Table 1: Notations

Notations	Descriptions
n	Number of categories for WF classification
L	Burst sequence length of the sample
ϕ^{WF}	WF classification model
$\boldsymbol{x}$	Pure data
$\boldsymbol{x}_a$	Adversarial sample with added adversarial noise
$\boldsymbol{e}$	Conditional embedding of the sample
k	Embedding dimension in the diffusion model
s	Frequency scaling factor of the trigonometric family used for time embedding
N	Number of discretization steps in the diffusion process
t_*	Time step for purifying diffusion
τ	Guidance weight for the WF model
β_0, β_1	Variance scheduler
N_{epoch}	Number of training iterations

3.1 Threat Model

In a black-box attack scenario against an NTP method, the adversary can only collect network flow data before and after the protection method is applied, without access to specifics of the implementation. Moreover, if NTP employs a neural network as its foundation, the model architecture and parameters themselves cannot be obtained by attackers. The black-box assumption precludes the extraction of gradient information or other insights that could compromise the defense or facilitate more potent attacks. Under this constrained view, the protections are a black box to would-be adversaries, limiting their ability to precisely reverse engineer vulnerabilities or craft evasions finely tuned to specific weaknesses.

3.2 Data Preprocess

Data preprocessing is performed by removing any network flow with fewer than 50 network data packets or those that begin with server-to-client data packets, considering them as invalid. Similar to previous work [8, 19],the sequence of network flows is transformed into a burst sequence, and the sequence is padded with zeros or truncated to a fixed length (L). A burst refers to the length of the longest consecutive substring in the same direction (i.e., same sign), As shown in Figure 2 , given a network flow sequence +1,+1,-1,-1,-1,-1,+1,-1,-1,-1,-1,+1, +1, -1, -1, -1, -1, +1, -1, -1, -1, -1 into a burst sequence is +2,-4,+1,-3,+2, -4, +1, -3.

3.3 UNetFormer

Based on [33], it has been demonstrated that appropriate selection of the time step t^* and the use of diffusion models can effectively remove adversarial noise from data. To this end, we propose DTPN, which leverages a meticulously designed UNetFormer network as the score estimator in the diffusion model training process. Drawing inspiration from the UNet architecture, UNetFormer incorporates one-dimensional convolutions of various sizes and a re-designed Transformer that better suits our task, as illustrated in Figure 3. UNetFormer comprises three components: DownBlock, TransformerBlock, and UpBlock. C, K, S, and G denote the output channel number of the convolution, convolution kernel size, convolution stride, and the number of groups in group normalization, respectively.

3.3.1 DownBlock. Assuming that the embedded space has a dimension of k, we begin by using the Gaussian Fourier mapping to perform a linear transformation for time step t, resulting in a time embedding $\mathbf{t}' \in \mathbb{R}^k$. The transformation is expressed as follows:

$$\mathbf{t}' = \sigma\left(\boldsymbol{W}^{(1)}[\sin(2\pi t \boldsymbol{W}^{(f)}) \| \cos(2\pi t \boldsymbol{W}^{(f)})]\right) \tag{1}$$

,where $\boldsymbol{W}^{(f)} \sim \mathcal{N}(0, s\boldsymbol{I}) \in \mathbb{R}^{k/2}$ is a frequency factor that is randomly initialized and not learnable, and $\boldsymbol{W}^{(1)} \in \mathbb{R}^{k\times k}$ is a learnable parameter matrix. The notation $\|$ denotes vector concatenation operation and the activation function $\sigma(x)$ is defined as $\mathrm{silu}(x) = \frac{x}{1+\exp(-x)}$.

Each DownBlock, which is a convolutional block, is denoted by the following equation:

$$\mathrm{DownBlock}\left(\mathbf{h}, \mathbf{t}'\right) = \sigma\left(\mathrm{Group\,Norm}\left(\mathrm{Conv1d}(\mathbf{h}) + \mathrm{Dense}\left(\mathbf{t}'\right)\right)\right) \tag{2}$$

,where Conv1d represents a one-dimensional convolution operation. $\mathrm{Dense}(\mathbf{t}') = \boldsymbol{W}^{\theta}\mathbf{t}' + \boldsymbol{b}^{\theta}$ represents a fully connected layer with $\boldsymbol{W}^{\theta} \in \boldsymbol{R}^{C\times k}$ and $\boldsymbol{b}^{\theta} \in \mathbb{R}^{C}$ are learnable parameters. Since the output shape of a one-dimensional convolution is $\mathbb{R}^{C\times L}$ while the output shape of a fully connected layer is $\mathbb{R}^{C}$, the output of the fully connected layer is broadcasted (copied) L times in the second dimension to become $\mathbb{R}^{C\times L}$ when added together.

GroupNorm normalizes each element x in the input data, as shown in the following equation (3):

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}}\gamma + \beta \tag{3}$$

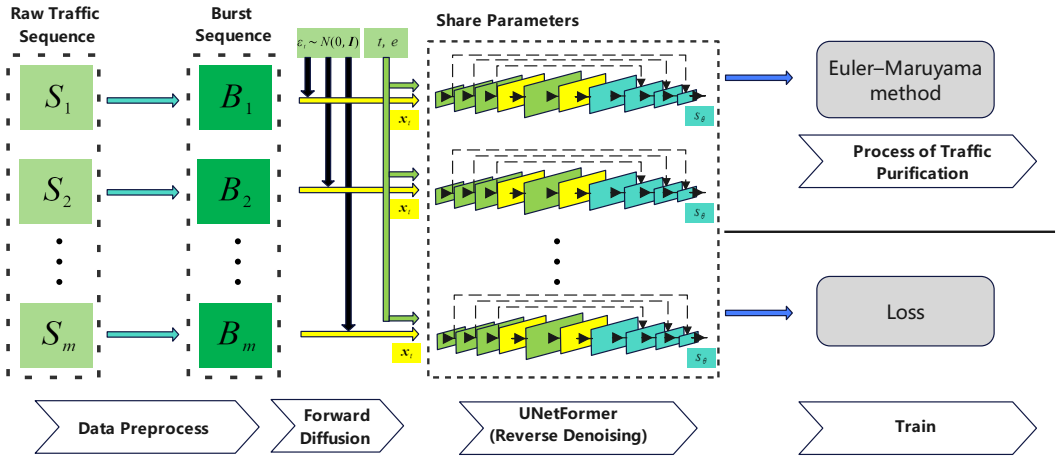


Figure 1: DTPN Model Architecture. Black arrows add Gaussian noise, deep green arrows add diffusion time steps and the penultimate layer's output from the WF model.The cyan, yellow, and blue arrows represent tensor flow.

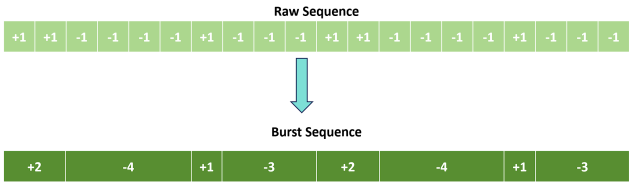


Figure 2: Converting a network flow to a burst sequence.

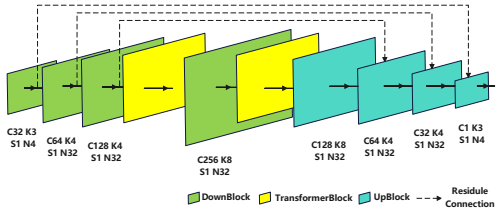


Figure 3: The schematic illustration of UNetFormer.

,where γ and β are learnable and shared within a single channel. To avoid numerical issues, a very small number ϵ is used, typically set to 10^{-5}. GroupNorm partitions the C channels of the input data into N groups, where the mean and variance are calculated for each group as $\mathbb{E}[x]$ and $\mathrm{Var}[x]$.

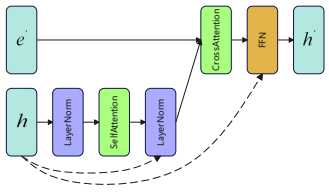


Figure 4: The schematic illustration of TransFormerBlock.

3.3.2 TransformerBlock. The TransformerBlock, illustrated in Figure 4, comprises SelfAttention module, CrossAttention module, Layer normalization (LayerNorm), and Feed-Forward Neural Network (FFN), with residual connections [34] indicated by dashed lines. We employ the trained DF model φ^{WF} to input x_0 and obtain the conditional embedding of the sample, $\boldsymbol{e}$, in the penultimate layer of the DF model. The conditional vector $\mathbf{e}'$ is obtained after an affine transformation, as presented in the following equation:

$$\boldsymbol{e}' = \boldsymbol{W}^{\boldsymbol{e}}\boldsymbol{e} + b^{\boldsymbol{e}} \tag{4}$$

,where $\boldsymbol{W}^{e} \in \mathbb{R}^{k\times m}$ and $\boldsymbol{b}^{e} \in \mathbb{R}^{k}$ are learnable parameters.

LayerNorm is defined similarly to Equation (3), but the mean and variance are computed from the last dimension (i.e., the dimension where the feature dimension K is located) of the input tensor. The FFN module is defined as shown in the following equation (5):

$$\mathrm{FFN}(\mathbf{h}) = \boldsymbol{W}^{(2)}\,\mathrm{gelu}\left(\boldsymbol{W}^{(1)}\mathbf{h} + \boldsymbol{b}^{(1)}\right) + \boldsymbol{b}^{(2)} \tag{5}$$

,where $\boldsymbol{W}^{(1)} \in \mathbb{R}^{3k\times k}, \boldsymbol{b}^{(1)} \in \mathbb{R}^{3k}, \boldsymbol{W}^{(2)} \in \mathbb{R}^{k\times 3k}$ and $\boldsymbol{b}^{(2)} \in \mathbb{R}^{k}$ are learnable parameters, activation function $\mathrm{gelu}(x) = x\Phi(x)$, $\Phi(x)$ is the cumulative distribution function of the Gaussian distribution.

3.3.3 UpBlock. Each UpBlock, which represents the deconvolutional block, can be represented by the following equation:

$$\mathrm{UpBlock}\left(\mathbf{h}, \mathbf{t}'\right) = \sigma\left(\mathrm{GroupNorm}\left(\mathrm{TransConv1d}(\mathbf{h}) + \mathrm{Dense}\left(\mathbf{t}'\right)\right)\right) \tag{6}$$

,where TransConv1d denotes a one-dimensional deconvolution operation.

3.4 Training DTPN

To train DTPN, we adopt the training method proposed by Song et al. in VP-SDE [35]. During the training phase, a random time step $t \in [0, 1]$ is sampled, and the variance scheduler β_t of the noise is adjusted to obtain the corresponding noise variance at the current time step. Subsequently, the data $\mathbf{x}_t$ with noise at the t-th time step during the forward diffusion process, was combined with the time step t and the conditional embedding e, and fed into the score estimator network $\boldsymbol{s}_\theta$ to estimate the noise. The underlying principle is that diffusion models based on both Bayesian theory and score models (i.e., SDE) have a common essence. The Bayesian-based diffusion model utilizes neural networks to estimate the noise ϵ_t at the t-th time step, whereas the SDE-based diffusion model uses neural networks to predict $-\frac{\epsilon_t}{\sqrt{1-\bar{\alpha}_t}}$, differing only by a coefficient. Thus, the mean squared error between the estimator network output

and the noise can serve as the first loss function, which can be formulated as follows:

$$\mathcal{L}_1 = \|S_\theta\left(\boldsymbol{x}_t, t, \boldsymbol{e}\right) + \frac{\epsilon_t}{1 - e^2\sigma_t}\|_2^2 \tag{7}$$

Given that our objective is attacks against NTP methods that rely on adversarial perturbations in the WF classification task, we require a well-trained WF model to guide the training process of the diffusion model. The guidance component can be expressed using the second loss function, which can be formulated as follows:

$$\mathcal{L}_2 = CE\left(y, \phi^{\mathrm{WF}}\left(\boldsymbol{x}_t - \sigma_t^2 \boldsymbol{s}_\theta\left(\boldsymbol{x}_t, t, \boldsymbol{e}\right)\right)\right) \tag{8}$$

,where CE(·) denotes the cross-entropy function.

The total loss is:

$$\mathcal{L}_{total} = \mathcal{L}_1 + (1 - t)\tau\mathcal{L}_2 \tag{9}$$

,where τ is a scalar hyperparameter.

To summarize, the workflow of the training phase of DTPN is shown in Algorithm 11. As for the optimization algorithm, we adopt the Adam [36] optimizer with the batch size B.

Algorithm 1 Workflow of the Training Phase

Input: Variance scheduler β_0 and β_1, the trained WF model ϕ^{WF}, WF model guiding weight τ, score estimator parameters θ, learning rate λ
Output: Trained score estimator parameters θ

$\beta_t = \beta_0 + t\left(\beta_1 - \beta_0\right)$
Initial model parameters: $\theta \leftarrow \theta_0$
while θ not converged **do**
  $(\mathbf{x}_0, y) \leftarrow (\mathbf{x}_a, y) \sim q\left(\mathbf{x}_\mathbf{a}\right)$
  $t \sim \mathcal{N}(0, 1)$
  $\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \boldsymbol{I}) \in \mathbb{R}^{1\times L}$
  $\sigma_t \leftarrow \exp\left(-\frac{1}{4}t^2\left(\beta_1 - \beta_0\right) - \frac{1}{2}t\beta_0\right)$
  $\boldsymbol{x_t} \leftarrow \sigma_t \boldsymbol{x}_0 + \left(1 - e^2\sigma_t\right)\boldsymbol{\epsilon}_t$
  $\mathcal{L}_1 \leftarrow \|S_\theta\left(\boldsymbol{x}_t, t, \boldsymbol{e}\right) + \frac{\epsilon_t}{1-e^2\sigma_t}\|_2^2$
  $\mathcal{L}_2 \leftarrow CE\left(y, \phi^{\mathrm{WF}}\left(\boldsymbol{x}_t - \sigma_t^2 s_\theta\left(\boldsymbol{x}_t, t, \boldsymbol{e}\right)\right)\right)$
  $\mathcal{L}_{total} = \mathcal{L}_1 + (1 - t)\tau\mathcal{L}_2$
  $\theta \leftarrow \theta - \lambda \cdot \mathrm{Adam}(\nabla_\theta \mathcal{L}_{total})$
end while

3.5 Traffic Purification

After acquiring the score estimator S_θ during the training phase, to obtain purified traffic $\hat{\boldsymbol{x}}(0)$ from the adversarial samples $\boldsymbol{x}_a$, a numerical SDE solver is required to obtain the numerical solution of SDE in the reverse diffusion process since the output of the trained score estimator is a score, which is the gradient of the data distribution. Therefore, the general steps for adversarial data purification include initializing the adversarial data as the initial data (i.e., $t = 0$), adding noise via the forward diffusion process, using the Euler-Maruyama [37] method to solve the reverse SDE for the trained score estimator, and finally obtaining the purified Traffic. Algorithm 2 provides a detailed purification process.

Algorithm 2 Adversarial Data Purification Process

Input: Adversarial Data $\boldsymbol{x}_a$, trained score estimator $\boldsymbol{S}_\theta$, Variance scheduler β_0 and β_1, time step for purifying diffusion $t^* \in [0, 1]$, time stride $\Delta t < 0$ with $\Delta t \approx 0$
Output: purified Traffic $\hat{\boldsymbol{x}}(0)$

$\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \boldsymbol{I}) \in \mathbb{R}^{1\times L}$
$t \leftarrow t^*$
$\sigma_t \leftarrow \exp\left(-\frac{1}{4}t^2\left(\beta_1 - \beta_0\right) - \frac{1}{2}t\beta_0\right)$
$\boldsymbol{x}_t \leftarrow \sigma_t \boldsymbol{x}_0 + \left(1 - e^2\sigma_t\right)\boldsymbol{\epsilon}_t$
$\beta(t) \leftarrow \beta_0 + t\left(\beta_1 - \beta_0\right)$
repeat
  $z_t \sim \mathcal{N}(0, \boldsymbol{I}) \in \mathbb{R}^{1\times L}$
  $\Delta x \leftarrow \sqrt{\beta(t)\Delta t} z_t - \left[\frac{1}{2}\beta(t)x_t + \beta(t)\boldsymbol{s}_\theta\left(x_t, t, e\right)\right]\Delta t$
  $x_t = x_t + \Delta x$
  $t \leftarrow t + \Delta t$
until $t \leqslant 0$
$\hat{\boldsymbol{x}}(0) \leftarrow \boldsymbol{x}_t$
return $\hat{\boldsymbol{x}}(0)$

4 EXPERIMENTS

4.1 Dataset and Experimental Settings

In this paper, we used two publicly available large-scale datasets, i.e., the DF dataset [8] and the CW100 dataset [38]. The DF dataset comprises 95 categories of websites, with each website having 1,000 network flows. The dataset was collected by visiting the homepages of all the top 100 Alexa sites for 1,250 times. The CW100 dataset consists of 100 different categories of websites, with each website containing approximately 2,500 network flows. In both the DF and CW100 datasets, each network flow (i.e., each data sample) is a sequence composed of +1 and 1, representing network data packets sent between the client and server. Subsequently, we perform data preprocessing, all the 95 websites in the experiment are used with 460 network flows from each website. Similarly, after preprocessing the CW100 dataset, websites with only 54 network flows in a category are removed, and 2,425 network flows from each of the remaining websites are selected for the experiment. Based on statistical analysis and model performance, we choose the network flows (samples) length of 512.

Additionally, we refer to the existing work on web traffic protection and use DF as the default WF model ϕ^{WF}. The DF model used in this paper is provided in [8]. This approach is designed for black-box attack scenarios, and it necessitates the collection of network flow data both before and after the utilization of the NTP (Network Time Protocol) technology. Specifically, this involves gathering the pure data $\boldsymbol{x}$ and the adversarial samples $\boldsymbol{x}_a$ to construct a dataset $\{(\boldsymbol{x}, \boldsymbol{x}_a, y)\}$. We implement the model using PyTorch, and all the experiments are conducted on a server with 128 GB of memory and four NVIDIA 3090 GPUs. In the specific implementation, the number of discretization steps $N = 1,000$ for the diffusion process, the step length $\Delta t = -\frac{1}{N}$ for the antagonistic data purification process, the number of training iterations $N_{\mathrm{epoch}} = 50$, and batch size = 512 are set.

Table 2: Network traffic purification experimental results(accuracy) in the DF dataset

Defense methods	WF test	Adversarial training	Our method
Walkie-Talkie [15]	40.0%	46.0%	**60.1%**
WTF-PAD [13]	27.5%	86.2%	**92.7%**
Mockingbird [17]	3.0%	33.3%	**66.3%**
AdvTraffic-AdvGAN [19]	1.9%	10.2%	**71.0%**

Table 3: Network traffic purification experimental results(accuracy) in the CW100 dataset

Defense methods	WF test	Adversarial training	Our method
Walkie-Talkie [15]	30.6%	41.8%	**57.7%**
WTF-PAD [13]	23.9%	85.1%	**90.4%**
Mockingbird [17]	7.4%	23.8%	**63.3%**
AdvTraffic-AdvGAN [19]	5.8%	24.9%	**75.2%**

4.2 Experiments on Adversarial Training under Different NTP Methods

In this experiment, we conduct experiments on four mainstream and state-of-the-art network traffic defense methods. Walkie-Talkie [15] and WTF-PAD [13] represent the two mainstream traffic obfuscation-based NTP methods. Mockingbird [17] and AdvTraffic-AdvGAN [19] are published in 2021 and 2022, respectively. They use a deep learning-based approach as an adversarial perturbation-based method for NTP.

We first evaluate the attack performance on defended data, where the DF model is trained on non-defended data and tested on defended data. The resulting accuracy is referred to as WF test accuracy. Next, we conduct adversarial training, where the DF model is both trained and tested on defended data. This experimental setup reflects a more realistic scenario in which a website fingerprinting attacker identifies the applied defense and retrains its classifier using defended data. The resulting accuracy is referred to as adversarial training accuracy. Finally, we apply our method (DTPN) by using defended data as input and purifying this data. The output is purified data. We then use the purified data as the dataset to test the trained DF model, and the resulting accuracy is referred to as our method accuracy. The dataset is split into 70% training, 10% validation, and 20% testing.

As shown in Tables 2 and 3 , Walkie-Talkie [15] has the lowest classification accuracy after traffic purification on the DF and CW100 datasets, with only 60.1% and 57.7% respectively, which is because Walkie-Talkie adds 149.6% bandwidth overhead to the network traffic, resulting in the loss of a large number of global discriminative features on the defensed network traffic. For the latest study, AdvTraffic-AdvGAN [19], the method is capable of effectively reducing the impact of adversarial training. As a result, even after retraining the network traffic data protected by AdvTraffic-AdvGAN using adversarial training, the classification accuracy of WF attackers on the DF and CW100 datasets can only reach 10.2% and 24.9%, respectively. This is basically unsuccessful for the WF classification task to identify user behaviors from the network traffic generated during the use of the onion network. Using the diffusion model-based network traffic purification method, the WF classification accuracy can be improved to 71.0% and 75.2% on the DF and CW100 datasets, respectively, making previously used attack methods usable again. In summary, compared to adversarial training, the maximum accuracy improvement achieved is 60.8%(71.0%-10.2%) and 50.3%(75.2%-24.9%) on the two datasets, respectively.

Table 4: Experimental results(accuracy) on different WF models

WF model	WF test	Adversarial training	Our method
DF [8]	1.9%	10.2%	**71.0%**
AWF [39]	1.1%	8.7%	**54.0%**
Var-CNN [9]	2.1%	11.4%	**61.1%**

4.3 Experiments on Different WF Models

Since the DF model was used as the WF model for all the experiments in the previous experiments in this chapter, in order to demonstrate the good migration of the method proposed in this paper, two other WF models, namely the AWF [39] and Var-CNN [9] models, are used for the experiments in this subsection. For the hyperparameters of the above two models, the settings in the original paper are used. And all the three experiments use AdvTraffic-AdvGAN [19] as the NTP method. For Var-CNN, since its original version also utilizes timestamp features, and there are no relevant features in the dataset used in this work, its timestamp-related part is removed in this experiment.

From Table 4 , it can be seen that the present method achieves better results than the adversarial training in all the three WF models. The DF model achieves the best performance among the three models, while the AWF model achieves the worst. AWF is the first and the weakest model among the three models, with only 147K training parameters, while DF and Var-CNN have 3,979K and 3,893K parameters, respectively, and therefore their performance is the worst in this method. Although Var-CNN performs better than DF in the adversarial training setting, the DF model performs better than Var-CNN in the setting of the present method. This is because Var-CNN has a more complex network structure (e.g., using inflated causal convolution), and performs better in adversarial training, although the model parameters are less than DF. However, for the network traffic purification based on the diffusion model, the results are worse. This is likely because the network structure of Var-CNN is too complex and significantly different from UNet used in this method. Additionally, the original Var-CNN requires timestamp-related features, which are not modeled in this method. As a result, Var-CNN's performance in the diffusion model is inferior to that of DF.Overall, the method still achieves significant improvement over the adversarial training, demonstrating its good transferability to different WF models.

4.4 Ablation Study

The diffusion model proposed in this paper for network traffic purification mainly has the following innovations compared to the original diffusion model: (a) the hidden layer features of the WF classification model are input to the diffusion model as the conditional embedding of the diffusion model; (b) the loss function of the WF classification model is introduced as an auxiliary task for the training of this method. Also for the UNetFormer used in the diffusion model as a fractional network, the hyperparameters of the UNetFormer network applied in this method are adopted similarly to the convolutional hyperparameters of the DF model, so that the global features of the network traffic can also be learned better. This subsection includes three experiments to showcase the

Table 5: The ablation experiment result

Variant	WF test accuracy
w/o conditional embedding	60.1%
w/o WF model auxiliary task	52.1%
Replace UNetFormer with MLP	29.6%
Our method	**71.0%**

effectiveness of the method proposed, which are (a) removing the conditional embedding, (b) removing the WF model auxiliary task, and (c) replacing the UNet with a three-layer MLP.

From Table 5 , we can see that the introduction of these three modules can significantly improve the effect of network traffic purification, among which the introduction of the WF model auxiliary task and UNetFormer has more obvious improvement on the effect of network traffic purification. This is because the WF model auxiliary task serves as regularization term. It can constrain the model, allowing the diffusion model to better capture the global features of network traffic rather than overfitting to the noise in the training data, thereby improving the model's generalization. UNetFormer, as a fractional network, utilizes a convolutional neural network architecture similar to that of DF to better learn the local features of the network traffic, thus enabling the diffusion model to better learn the features that can improve the robustness of the WF recognition model. In contrast, replacing UNetFormer with a three-layer MLP, the effectiveness of network traffic purification is significantly reduced because MLP cannot learn the local features of network traffic. Additionally, Even for the least effective module, the hidden layer features of the WF model, is used as the conditional embedding of the conditional UNetFormer network, and the performance improvement is significant, with a gain of 10.9 percentage points.

4.5 DTPN Hyperparameter Selection Experiments

(1) Learning Rate

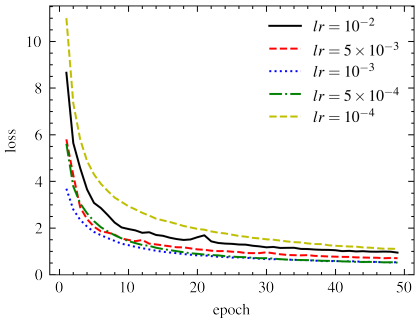


Figure 5: Test loss curves for different learning rates

Because the neural network model uses a stochastic gradient descent optimizer, each training session will feed data into the model in the form of batches and obtain loss values and corresponding gradient information, which is then used to update the parameters of the model. At each update, the parameter that controls the magnitude of the update is called the learning rate. Different learning rates can affect how quickly a model converges during training and lead to different final model performances. Additionally, the optimal learning rate varies depending on the model, data, and loss function. Therefore, it is not possible to determine the optimal learning rate through theoretical analysis alone; this will be explored experimentally.

As shown in Figure 5 , it can be seen that as the learning rate decreases, the loss value becomes smoother throughout the training process of the model, and the final loss value is also smaller. In summary, 10^{-3} is chosen as the learning rate used in the subsequent experiments in this paper.

(2) Feature dimension k

Since UNetFormer is used as the backbone network, in which the embedding dimension of the time step, the conditional vector after affine transformation, the convolution, and the dimension of attention are all in a uniform feature dimension k. The optimal solution of hyperparameter k cannot be derived theoretically. An overly large k will result in a model with too many parameters and a larger hypothesis space, which in turn makes the model more prone to overfitting, reduces its generalization performance, and increases the training time. A too small k may lead to an underfitting problem because the model is not expressive enough to fit the data well. For each value of feature dimension, 30 cycles of training will be performed, and the training loss and test accuracy of the last cycle will be recorded.Table 6 shows that test accuracy reaches the optimal value at k=128, and then tends to decrease as k continues to increase. However, after the feature dimension 128, the test accuracy of the model does not significantly decrease. While the training loss reaches its optimal value at k=512, the test accuracy is not at its best, which may be due to overfitting. Therefore, a feature dimension of k=128 is chosen as the optimal value for subsequent experiments.

Table 6: Training loss and test accuracy for different feature dimensions k

k	Training Loss	Test Accuracy
8	0.340	0.581
16	0.311	0.601
32	0.202	0.672
64	0.190	0.698
128	0.181	**0.708**
256	0.173	0.701
512	**0.168**	0.697
1024	0.298	0.617

(3) Variance scheduler β_0 and β_1

The variance scheduler parameters β_0 and β_1 are used as hyperparameters of the variance scheduler to control the amount of noise added to the data during the forward diffusion process. In addition, a linear variance scheduling function is used. For the noise variance at time step t, it is calculated as $\beta_t = \beta_0 + t(\beta_1 - \beta_0)$, where β_0 and β_1 denote the noise variance at the beginning and end of the forward diffusion moments, respectively, and $\beta_0 < \beta_1$. If the variance scheduling factor is taken too small, it will cause both the training and inference process of the model to use more diffusion time steps for the malicious semantic information of the counteracting noise to be completely eliminated. If the variance scheduling factor is too large, it may cause the model to add too much noise to the data during the forward diffusion process, making too much

local semantic information of the data lost, and thus the fractional network model cannot converge well.

Table 7: Training loss and test accuracy for different variance scheduling factors

β_0	β_1	Training Loss	Test Accuracy
0.0	0.1	0.375	0.561
0.0	0.025	0.197	0.689
0.0	0.001	0.245	0.623
0.0002	0.1	0.369	0.566
0.0002	0.025	**0.183**	**0.709**
0.0002	0.001	0.228	0.641
0.001	0.1	0.479	0.499
0.001	0.025	0.303	0.588

Each set of parameters is trained for 50 epochs, and the training loss and test accuracy of the final epoch are recorded. As shown in Table 7 , from the two sets of experimental results, it can be observed that as long as β_0 is kept at a relatively low value, its minor fluctuations have little impact on the performance of the diffusion model. In summary, we choose $\beta_0 = 0.0002$ and $\beta_1 = 0.025$ as the optimal variance scheduling factors for the diffusion model.

(4) Guidance weight τ for the WF model

When applying the inverse diffusion process to recover adversarial data with added diffusion noise, the diffusion model needs to use an existing WF recognition model to guide its output. Otherwise, the diffusion model cannot learn good feature information and thus cannot recover data that are not adversarial and can be correctly classified by the WF recognition model. In the experiments of this subsection, the WF recognition model used is the DF model, and its loss function used in the training process is the cross-entropy loss function. How to balance the loss function of the diffusion model itself and this auxiliary loss function needs to be adjusted using a weight τ. In this paper, the values of the WF recognition model guidance weight τ are taken from 0.0 to 40 in uniform intervals, and a total of 10 values are taken. From Figure 6 , it can be observed that as τ increases from 0 to 20, the test accuracy of the diffusion model steadily improves overall, although there is some minor fluctuation. However, beyond τ values of 32, there is a significant decrease in accuracy. Therefore, this paper adopts 20 as the final value for the WF model guidance weight τ.

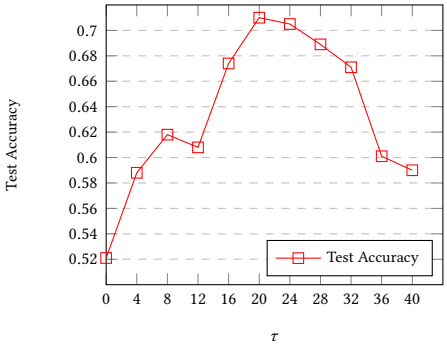


Figure 6: Test accuracy for different WF model guidance weights τ

(5) Optimal purification diffusion time step t^*

Based on Theorems 3.1 and 3.2 in [33], it becomes evident that local adversarial noise within adversarial data can be eradicated during the forward diffusion process and subsequently restored during the reverse diffusion process, leading to the generation of non-adversarial data. Additionally, clean data should not introduce excessive noise, as an excessively high level of noise can also eliminate the underlying global label semantics of the data. Therefore, it is necessary to find an optimal purification diffusion time step t^*. The optimal diffusion time step t^* is determined by the trade-off between purifying local perturbations (with larger t^*) and preserving global structures of adversarial data (with smaller t^*), to achieve high robust classification accuracy. From Figure 7 , it can be seen that when the optimal purification diffusion time step $t^* = 5\times10^{-3}$, the WF classification model ϕ^{WF} achieves the highest accuracy of 71.0%. As t^* increases, the accuracy of ϕ^{WF} decreases. Moreover, as t^* increases, the time consumed for data purification increases linearly, so we can use smaller t^* as much as possible to reduce the resource overhead of the purification process while ensuring the accuracy of ϕ^{WF}. Therefore, $t^* = 5 \times 10^{-3}$ is the optimal choice.

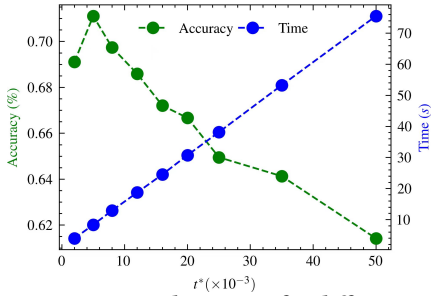


Figure 7: Test accuracy and test time for different purification diffusion time steps

5 CONCLUSION AND FUTURE WORK

As traffic protection methods based on GANs have emerged, adversarial training has proven to be ineffective in enhancing the adversarial robustness of WF models. To improve the performance of existing WF attack methods, we introduce diffusion models to purify adversarial data and counter NTP methods. Experimental results demonstrate substantial improvements achieved by the proposed approach against both mainstream and state-of-the-art NTP methods, surpassing the performance of adversarial training. Diffusion-based methods use reverse diffusion to recover clean data from noisy data. The noise addition involves smaller step size and numerous iterations of model inference, which can make it slower than methods such as GANs. It is potential to speed up network traffic purification.

6 Acknowledgments

This work was supported in part by Open Foundation of Key Laboratory of Cyberspace Security, Ministry of Education of China and Henan Key Laboratory of Network Cryptography (No.KLCS20240304), the Major Key Project of PCL(No. PCL2023A06-4), the Key project of Shenzhen municipality (No. JSGG20211029095545002), the Teaching Innovation Team of SZIIT, and the Major Key Project of PCL (No. PCL2023A05-4).

References

[1] Xiaochuan Gou and Xiangliang Zhang. Telecommunication traffic forecasting via multi-task learning. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*, pages 859–867, 2023.

[2] The Tor Project. The tor project. Website, 2023. Accessed on May 10, 2023.

[3] Dominik Herrmann, Rolf Wendolsky, and Hannes Federrath. Website fingerprinting: attacking popular privacy enhancing technologies with the multinomial naïve-bayes classifier. In *Proceedings of the 2009 ACM workshop on Cloud computing security*, pages 31–42, 2009.

[4] Andriy Panchenko, Lukas Niessen, Andreas Zinnen, and Thomas Engel. Website fingerprinting in onion routing based anonymization networks. In *Proceedings of the 10th annual ACM workshop on Privacy in the electronic society*, pages 103–114, 2011.

[5] Tao Wang, Xiang Cai, Rishab Nithyanand, Rob Johnson, and Ian Goldberg. Effective attacks and provable defenses for website fingerprinting. In *23rd {USENIX} Security Symposium ({USENIX} Security 14)*, pages 143–157, 2014.

[6] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz, and Rachel Greenstadt. A critical evaluation of website fingerprinting attacks. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, pages 263–274, 2014.

[7] Jamie Hayes, George Danezis, et al. k-fingerprinting: A robust scalable website fingerprinting technique. In *USENIX security symposium*, pages 1187–1203, 2016.

[8] Payap Sirinam, Mohsen Imani, Marc Juarez, and Matthew Wright. Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1928–1943, 2018.

[9] Sanjit Bhat, David Lu, Albert Kwon, and Srinivas Devadas. Var-cnn: A data-efficient website fingerprinting attack based on deep learning. *Proceedings on Privacy Enhancing Technologies*, 2019(4):292–310, 2019.

[10] Mohammad Saidur Rahman, Payap Sirinam, Nate Mathews, Kantha Girish Gangadhara, and Matthew Wright. Tik-tok: The utility of packet timing in website fingerprinting attacks. *arXiv preprint arXiv:1902.06421*, 2019.

[11] Xiang Cai, Rishab Nithyanand, and Rob Johnson. Cs-buflo: A congestion sensitive website fingerprinting defense. In *Proceedings of the 13th Workshop on Privacy in the Electronic Society*, pages 121–130, 2014.

[12] Xiang Cai, Rishab Nithyanand, Tao Wang, Rob Johnson, and Ian Goldberg. A systematic approach to developing and evaluating website fingerprinting defenses. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, pages 227–238, 2014.

[13] Marc Juarez, Mohsen Imani, Mike Perry, Claudia Diaz, and Matthew Wright. Toward an efficient website fingerprinting defense. In *Computer Security–ESORICS 2016: 21st European Symposium on Research in Computer Security, Heraklion, Greece, September 26-30, 2016, Proceedings, Part I 21*, pages 27–46. Springer, 2016.

[14] Rishab Nithyanand, Xiang Cai, and Rob Johnson. Glove: A bespoke website fingerprinting defense. In *Proceedings of the 13th Workshop on Privacy in the Electronic Society*, pages 131–134, 2014.

[15] Tao Wang, Ian Goldberg, et al. Walkie-talkie: An efficient defense against passive website fingerprinting attacks. In *USENIX Security Symposium*, pages 1375–1390, 2017.

[16] Chengshang Hou, Gaopeng Gou, Junzheng Shi, Peipei Fu, and Gang Xiong. Wf-gan: Fighting back against website fingerprinting attack using adversarial learning. In *2020 IEEE Symposium on Computers and Communications (ISCC)*, pages 1–7. IEEE, 2020.

[17] Mohammad Saidur Rahman, Mohsen Imani, Nate Mathews, and Matthew Wright. Mockingbird: Defending against deep-learning-based website fingerprinting attacks with adversarial traces. *IEEE Transactions on Information Forensics and Security*, 16:1594–1609, 2020.

[18] Milad Nasr, Alireza Bahramali, and Amir Houmansadr. Defeating dnn-based traffic analysis systems in real-time with blind adversarial perturbations. In *USENIX Security Symposium*, pages 2705–2722, 2021.

[19] Hao Liu, Jimmy Dani, Hongkai Yu, Wenhai Sun, and Boyang Wang. Advtraffic: Obfuscating encrypted traffic with adversarial examples. In *2022 IEEE/ACM 30th International Symposium on Quality of Service (IWQoS)*, pages 1–10. IEEE, 2022.

[20] Ding Li, Yuefei Zhu, Minghao Chen, and Jue Wang. Minipatch: Undermining dnn-based website fingerprinting with adversarial patches. *IEEE Transactions on Information Forensics and Security*, 17:2437–2451, 2022.

[21] Tao Wang and Ian Goldberg. Improved website fingerprinting on tor. In *Proceedings of the 12th ACM workshop on Workshop on privacy in the electronic society*, pages 201–212, 2013.

[22] Payap Sirinam, Nate Mathews, Mohammad Saidur Rahman, and Matthew Wright. Triplet fingerprinting: More practical and portable website fingerprinting with n-shot learning. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 1131–1148, 2019.

[23] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.

[24] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*, 2017.

[25] Florian Tramèr, Alexey Kurakin, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. Ensemble adversarial training: Attacks and defenses. *arXiv preprint arXiv:1705.07204*, 2017.

[26] Chaowei Xiao, Bo Li, Jun-Yan Zhu, Warren He, Mingyan Liu, and Dawn Song. Generating adversarial examples with adversarial networks. *arXiv preprint arXiv:1801.02610*, 2018.

[27] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, pages 2256–2265. PMLR, 2015.

[28] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019.

[29] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

[30] Yuhao Wang, Ziru Liu, Yichao Wang, Xiangyu Zhao, Bo Chen, Huifeng Guo, and Ruiming Tang. Diff-msr: A diffusion model enhanced paradigm for cold-start multi-scenario recommendation. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pages 779–787, 2024.

[31] Yu Liu, Yanan Cao, Shi Wang, Qingyue Wang, and Guanqun Bi. Generative models for complex logical reasoning over knowledge graphs. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pages 492–500, 2024.

[32] Weili Nie, Brandon Guo, Yujia Huang, Chaowei Xiao, Arash Vahdat, and Anima Anandkumar. Diffusion models for adversarial purification. In *International Conference on Machine Learning (ICML)*, 2022.

[33] Weili Nie, Brandon Guo, Yujia Huang, Chaowei Xiao, Arash Vahdat, and Anima Anandkumar. Diffusion models for adversarial purification. *arXiv preprint arXiv:2205.07460*, 2022.

[34] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[35] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.

[36] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.

[37] Desmond J Higham. An algorithmic introduction to numerical simulation of stochastic differential equations. *SIAM review*, 43(3):525–546, 2001.

[38] Vera Rimmer, Davy Preuveneers, Marc Juarez, Tom Van Goethem, and Wouter Joosen. Automated website fingerprinting through deep learning. *arXiv preprint arXiv:1708.06376*, 2017.

[39] Vera Rimmer, Davy Preuveneers, Marc Juarez, Tom Van Goethem, and Wouter Joosen. Automated website fingerprinting through deep learning. *Proceedings 2018 Network and Distributed System Security Symposium (NDSS)*, 2018.